

Grafos

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br





[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

27 de outubro de 2026

Roteiro

Grafos
- Objetivos
- Introdução
- Conceitos Fundamentais
- Terminologia
- Caminhos e Conectividade
- Grafos Direcionados e Ponderados

Estrutura de Árvore em Grafo



Roteiro

Representação computacional
- Matriz de Adjacências
- Matriz de Incidências
- Lista de Adjacências

Algoritmos de Busca
- Busca em Largura
- Busca em Profundidade
- Trabalho I.



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

- **Conceituar** o que é um grafo e identificar seus componentes fundamentais (vértices e arestas).
- **Diferenciar** os principais tipos de grafos e suas terminologias (simples, dígrafo, ponderado, etc.).
- **Modelar** problemas do mundo real utilizando a estrutura de grafos.
- **Implementar** as três principais representações computacionais de um grafo: Matriz de Adjacências, Lista de Adjacências e Matriz de Incidência.
- **Analisar** as vantagens e desvantagens de cada representação, sabendo escolher a mais adequada para cada cenário.
- **Aplicar** algoritmos de travessia fundamentais: Busca em Largura (BFS) e Busca em Profundidade (DFS).
- **Resolver** o problema do caminho mínimo com o Algoritmo de Dijkstra.



Grafos Estão em Todo Lugar!

- Pense em grafos como uma forma de representar **conexões**. Quase tudo no mundo pode ser visto como um conjunto de "coisas" e das relações entre elas.

Exemplos do Mundo Real

- **Redes Sociais:** você e seus amigos são os "pontos" (vértices), e as amizades são as "linhas" (arestas) que os conectam.
- **Mapas e GPS:** cidades são vértices e as estradas entre elas são arestas. O GPS usa algoritmos em grafos para encontrar o caminho mais rápido!
- **A Internet:** páginas da web são vértices, e os links de uma página para outra são as arestas.
- **Logística e Transportes:** aeroportos são vértices, e os voos diretos são arestas. Como uma empresa aérea otimiza suas rotas? Com grafos.

- A Teoria dos Grafos nos dá a linguagem e as ferramentas para analisar e resolver esses problemas de conexão de forma estruturada.



A Definição Formal de um Grafo

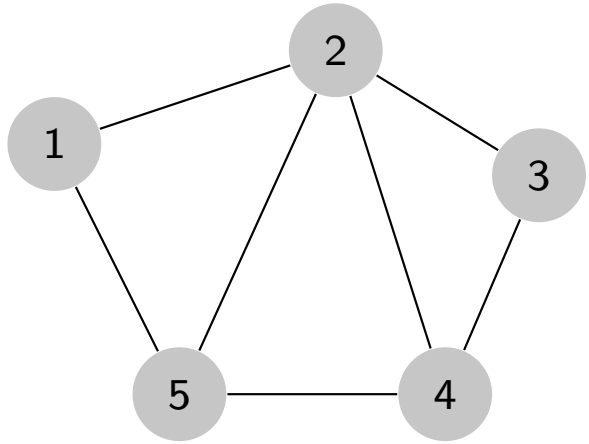
- Formalmente, um grafo **G** é um par ordenado $\mathbf{G} = (V, E)$, onde:
  - **V** é um conjunto de **Vértices** (ou nós). São os "pontos" ou as "entidades" do nosso modelo.
  - **E** é um conjunto de **Arestas** (ou arcos). São as "linhas" que conectam pares de vértices e representam a relação entre eles.

- Uma aresta $\mathbf{e} \in E$ que conecta os vértices u e v é representada como um par: $\mathbf{e} = (u, v)$.
  - Dizemos que os vértices u e v são **adjacentes** (ou vizinhos).
  - A aresta (u, v) é **incidente** aos vértices u e v.



Representação de um Grafo

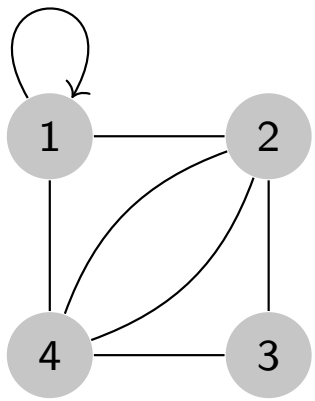
Vértices	$1, 2, 3, 4, 5$
Arestas	(1,2),(1,5),(5,2),(5,4), (2,4),(2,3),(4,3)
Tamanho $\|V\|$	Número de elementos em G
$\|E\|$	Número de arestas em G



Tipos Especiais de Arestas e Grafos

Laços e Arestas Paralelas

- **Laço (Loop):** uma aresta que conecta um vértice a si mesmo, ex: (v, v).
- **Arestas Paralelas (ou Múltiplas):** quando existe mais de uma aresta conectando o mesmo par de vértices.



Tipos Especiais de Arestas e Grafos

Classificações Importantes

- **Multigrafo:** um grafo que pode conter laços e/ou arestas paralelas.
- **Grafo Simples:** um grafo que **não** possui laços nem arestas paralelas. *(Este é o tipo mais comum que estudaremos!)*
- **Grafo Trivial:** um grafo com apenas um vértice e nenhuma aresta.

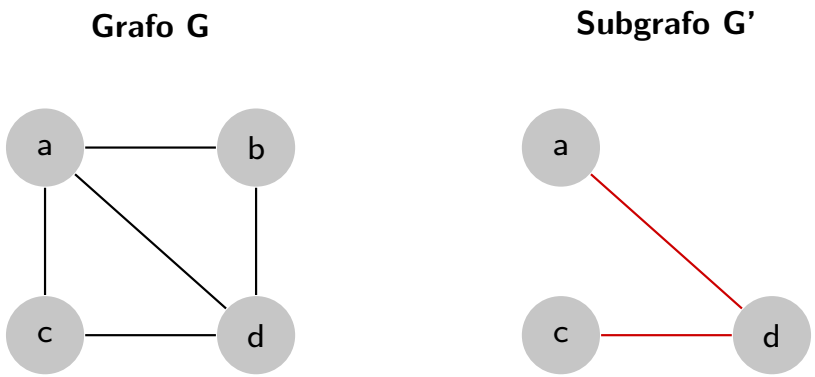


Subgrafo

Definição

Um grafo $G' = (V', E')$ é um **subgrafo** de $G = (V, E)$ se, e somente se, $V' \subseteq V$ e $E' \subseteq E$.

- Em outras palavras, um subgrafo é formado quando tomamos um "pedaço" dos vértices e das arestas do grafo original.
- **Exemplo prático:** a rede de amizades dos alunos de uma única turma é um subgrafo da rede de amizades de toda a universidade.

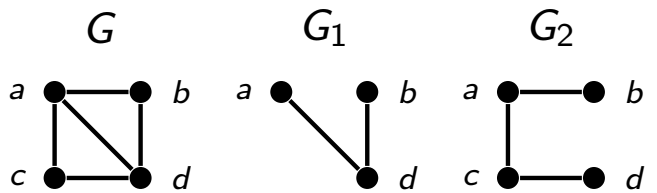


Subgrafo

G $V = a, b, c, d$
$E = \{ab, ac, ad, bd, cd\}$

G_1 $V_1 = a, b, d$
$E_1 = \{ad, bd\}$

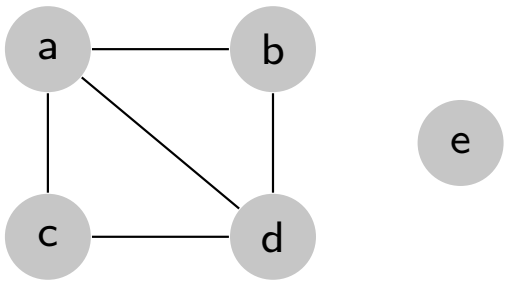
G_2 $V_2 = a, b, c, d$
$E_2 = \{ab, ac, cd\}$





Grau de um Vértice

- O **Grau** (ou valência) de um vértice v, denotado por $deg(v)$, é o número de arestas incidentes a ele. Em grafos com laços, cada laço conta 2 no grau.

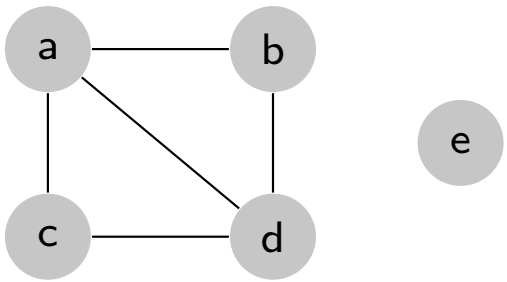


Exemplos de Grau

- $deg(a) = 3$
- $deg(b) = 2$
- $deg(d) = 3$
- $deg(e) = 0$

Grau de um Vértice

- O **Grau** (ou valência) de um vértice v, denotado por $deg(v)$, é o número de arestas incidentes a ele. Em grafos com laços, cada laço conta 2 no grau.



Terminologia

- **Vértice Isolado:** um vértice com grau 0.
- **Vértice Folha (ou Terminal):** um vértice com grau 1.

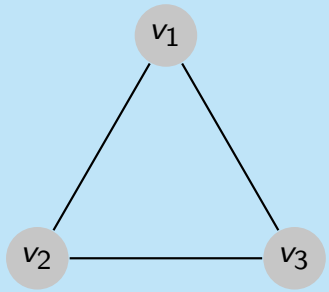
Theorem (Handshaking Lemma)

A soma dos graus de todos os vértices de um grafo é igual a duas vezes o número de arestas: $\sum_{v \in V} deg(v) = 2|E|$.

Grafos Regulares e Completos

Grafo Regular

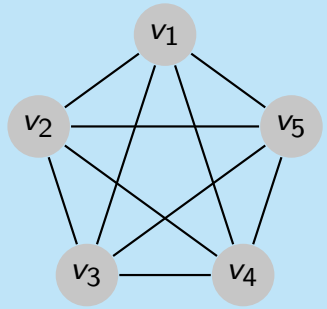
Um grafo é **k-regular** se todos os seus vértices possuem grau k.



Grafo 2-Regular

Grafo Completo (K_n)

Um grafo simples é **completo** se todo par de vértices distintos é conectado por uma aresta. O grafo completo com n vértices é denotado por K_n.



Grafo Completo K_5

Theorem

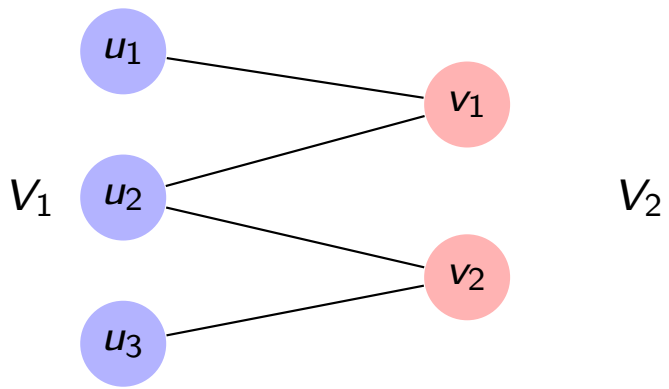
O número de arestas em um grafo completo K_n *é* $|E| = \frac{n(n-1)}{2}$.

Grafo Bipartido

Definição

- Um grafo $G = (V, E)$ é dito bipartido quando **V** pode ser particionado em dois subconjuntos V_1 e V_2 de modo que toda aresta de G liga um vértice de V_1 a um vértice de V_2

$$G(V_1 \cup V_2, E)$$

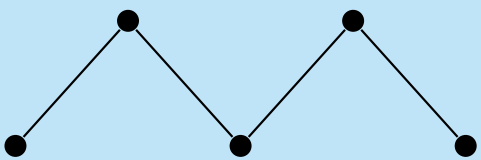


Grafo Bipartido Completo

Definição

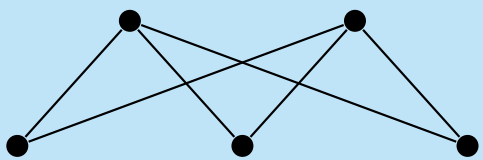
- Um grafo $G = (V_1 \cup V_2, E)$ é dito bipartido completo quando existe uma aresta para todo par de vértices u, v com $u \in V_1$ e $v \in V_2$
- $K_{m,n}$
  - $m = |V_1|$
  - $n = |V_2|$

(a) Grafo bipartido



(a)

(b) Grafo bipartido completo



(b) $K_{2,3}$

Caminhos e Ciclos

Definição

- Dado $G = (V, E)$, uma sequência de vértices $v_1, v_2, \ldots, v_j$, tal que $(v_i, v_{i+1}) \in E$ para $1 \leq i \leq j - 1$, recebe o nome de **caminho** de v_1 a v_j

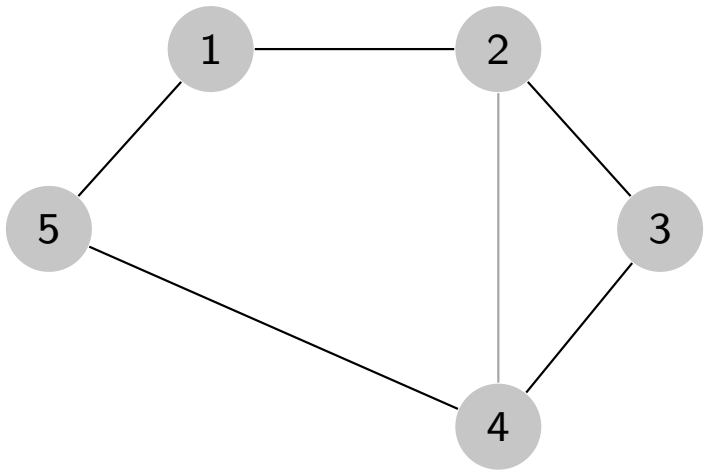
  Comprimento Número de arestas do caminho
  Caminho simples Todos os vértices são distintos
  Trajeto Todas as arestas são distintas



Caminhos, Trajetos e Ciclos (Parte 1)

Definições Iniciais: Como "Andar" no Grafo

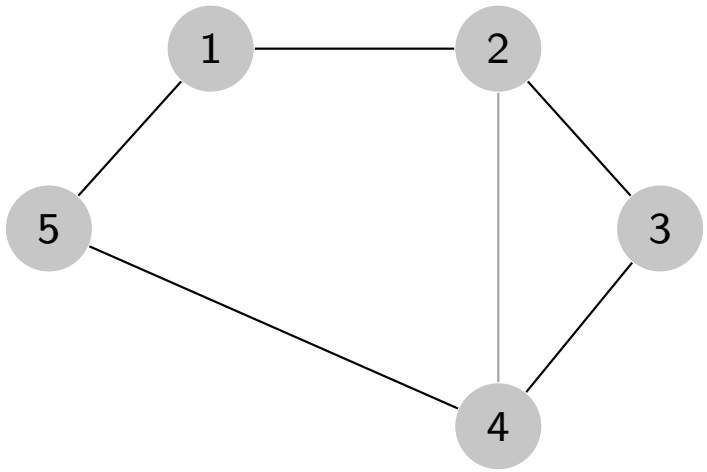
- **Caminho (Walk):** a forma mais geral de travessia. É uma sequência de vértices em que cada par consecutivo está conectado por uma aresta. **Vértices e arestas podem ser repetidos**. **Exemplo de Caminho (vértice 2 repetido):** $1 \to 2 \to 3 \to 4 \to 2$



Caminhos, Trajetos e Ciclos (Parte 1)

Definições Iniciais: Como "Andar" no Grafo

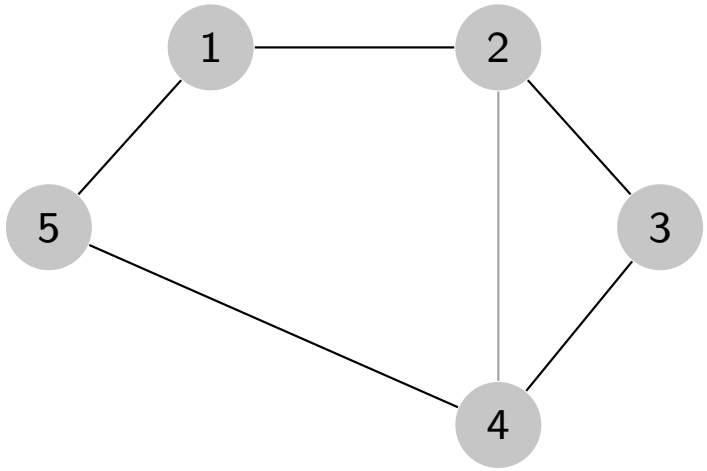
- **Trajeto (Trail):** um tipo especial de caminho em que as **arestas não se repetem**. Os vértices, por outro lado, ainda podem ser visitados mais de uma vez. **Exemplo de Trajeto (vértice 2 repetido):** $1 \to 5 \to 4 \to 2 \to 3$



Caminhos, Trajetos e Ciclos (Parte 2)

Definições Fundamentais: As Travessias Mais Úteis

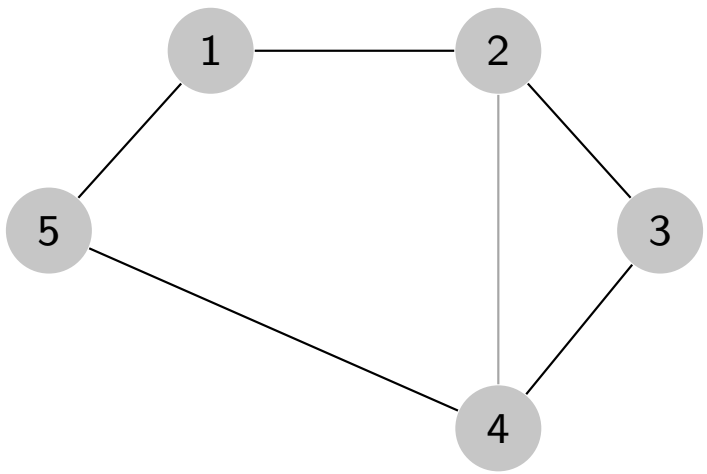
- **Caminho Simples (Path):** o conceito mais importante de "caminho". É um trajeto em que os **vértices não se repetem** (com exceção, talvez, do início e do fim). Ex: **Caminho Simples de 1 a 3:** $1 \to 2 \to 3$



Caminhos, Trajetos e Ciclos (Parte 2)

Definições Fundamentais: As Travessias Mais Úteis

- **Ciclo (Cycle):** um caminho simples de comprimento 3 ou mais que **começa e termina no mesmo vértice**. Ciclos são estruturas cruciais na análise de grafos. Ex: **Ciclo:** $2 \to 3 \to 4 \to 2$

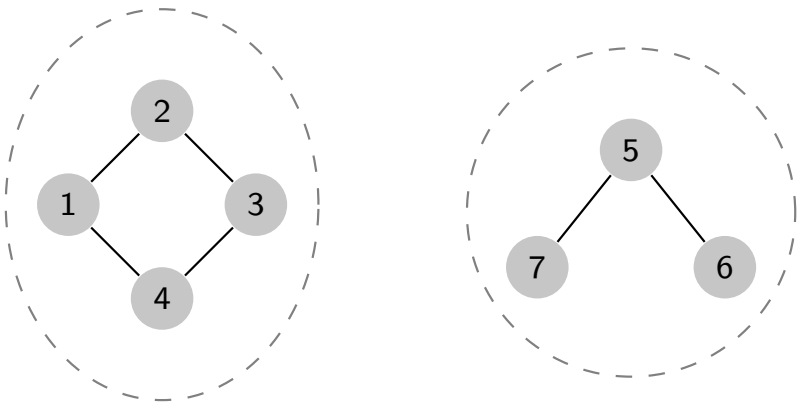


Conectividade

Grafo Conexo

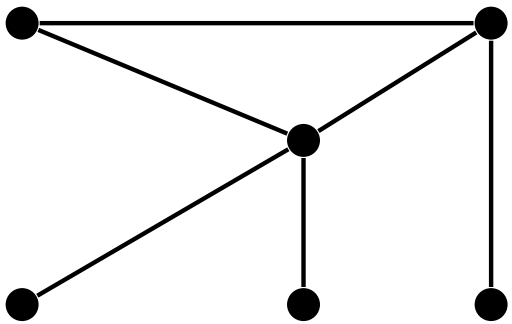
Um grafo (não-direcionado) é **conexo** se existe um caminho entre todo par de vértices distintos.

- Se um grafo não é conexo, ele é chamado de **desconexo**.
- Um grafo desconexo é composto por duas ou mais **componentes conexas**.
- **Analogia:** pense no mapa de um país com ilhas desconectadas. Cada ilha, com suas cidades e estradas, é uma componente conexa.



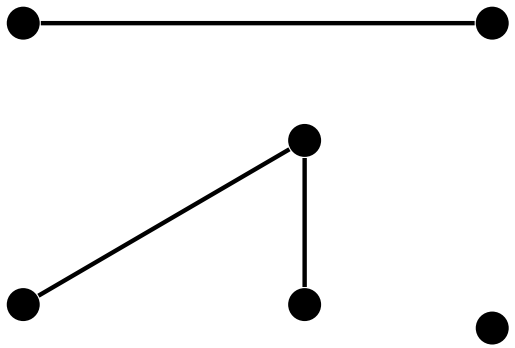
Grafo Conexo

(a) Grafo conexo



(a)

(b) Grafo desconexo

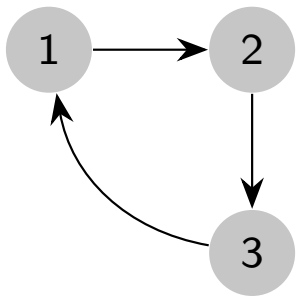


(b)



Grafos Direcionados (Dígrafos)

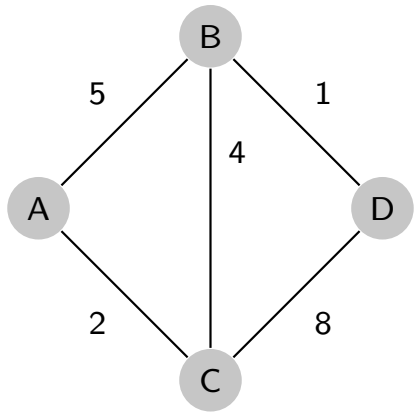
- Em um **Dígrafo**, as arestas (chamadas de **arcos**) têm uma direção. A aresta (u, v) indica que há uma conexão de u para v, mas não necessariamente de v para u.
- **Analogia:** ruas de mão única.
- **Terminologia Adicional:**
  - **Grau de Entrada (in-degree):** número de arcos que chegam a um vértice.
  - **Grau de Saída (out-degree):** número de arcos que partem de um vértice.



Vértice 2:
Grau de Entrada: 1
Grau de Saída: 1

Grafos Ponderados

- Em um **Grafo Ponderado**, cada aresta possui um valor numérico associado, chamado de **peso** ou **custo**.
- Este peso pode representar distância, tempo, custo financeiro, capacidade, etc.



Grafos Ponderados

- Em um **Grafo Ponderado**, cada aresta possui um valor numérico associado, chamado de **peso** ou **custo**.
- Este peso pode representar distância, tempo, custo financeiro, capacidade, etc.

Aplicações

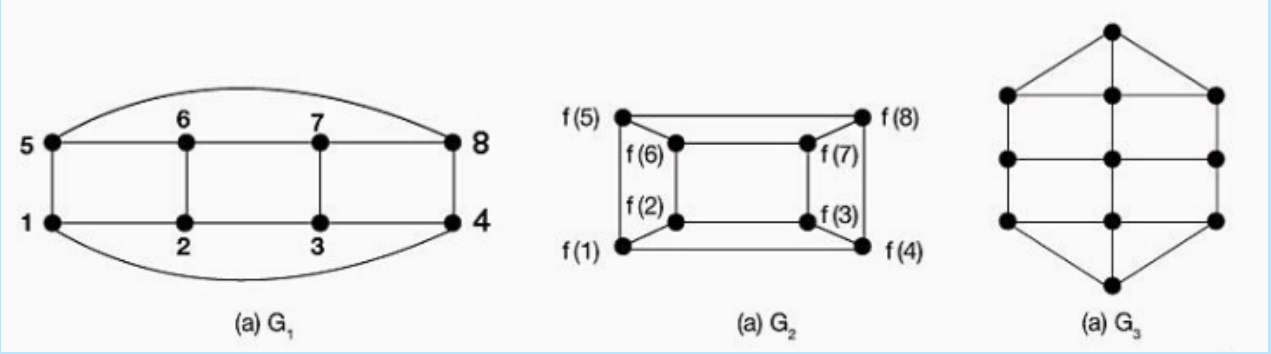
Grafos ponderados são essenciais para problemas de otimização. O Algoritmo de Dijkstra, que veremos mais tarde, opera em grafos ponderados para encontrar o caminho de menor custo.



Grafos Isomorfos

Definição

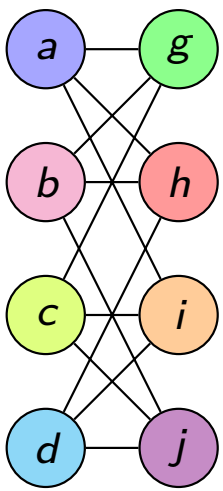
- Dois grafos $G_1(V_1, E_1)$ e $G_2(V_2, E_2)$ são isomorfos se existir uma bijeção $f : V_1 \rightarrow V_2$ tal que $(u, v) \in E_1$ se e somente se $(f(u), f(v)) \in E_2$
  - $(\mathbf{u}, \mathbf{v}) \in \mathbf{E_1} \iff (\mathbf{f(u)}, \mathbf{f(v)}) \in \mathbf{E_2}$
  - É possível re-rotular os vértices de G_1 com os rótulos de G_2 mantendo as arestas correspondentes em G_1 e G_2



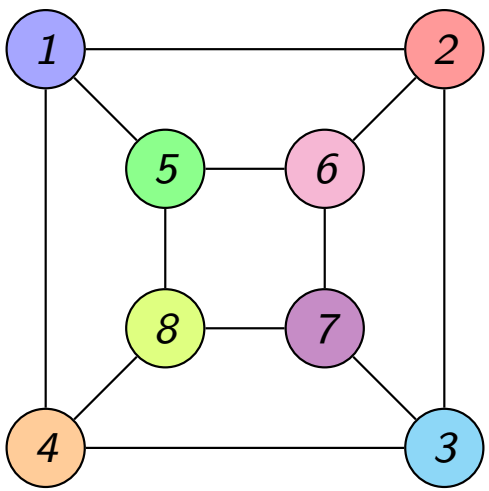
Grafos Isomorfos

- São aplicados em grafos com a mesma estrutura topológica

Grafo G



Grafo H



Isomorfismo entre G e H

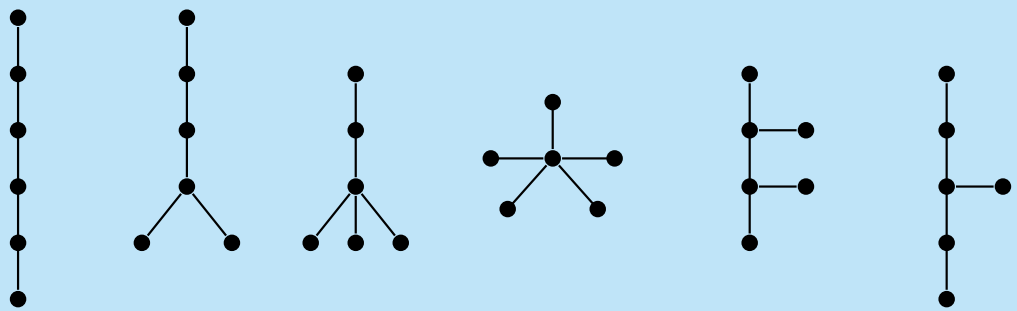
$f(a) = 1$
$f(b) = 6$
$f(c) = 8$
$f(d) = 3$
$f(g) = 5$
$f(h) = 2$
$f(i) = 4$
$f(j) = 7$



Árvore

Definição

- Um grafo $T(V, E)$ conexo e sem ciclos é chamado **árvore** e possui as seguintes propriedades:
  - Seja $v \in V$. Se v possui grau ≤ 1, então v é **folha**. Se o grau é > 1, então v é **interno**
  - Uma árvore T com n vértices possui $n - 1$ arestas
  - Um grafo G é uma árvore se e somente se existe um único caminho entre cada par de vértices de G

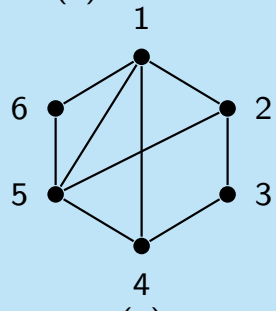


Árvore geradora

Definição

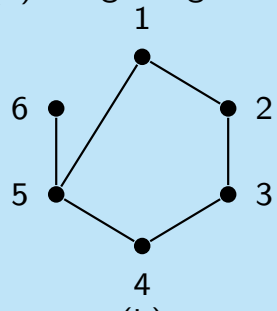
- Dado um grafo $G(V, E)$, denomina-se **subgrafo gerador** o grafo $H(V, E')$ que é subgrafo de G e tal que $V(G) = V(H)$
  - Se H é uma árvore, então é chamado de **árvore geradora**

(a) Grafo G



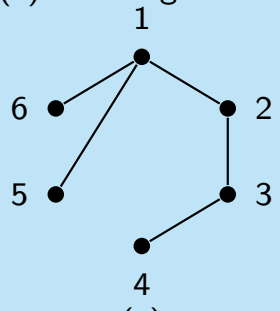
(a)

(b) Subgrafo gerador



(b)

(c) Árvore geradora

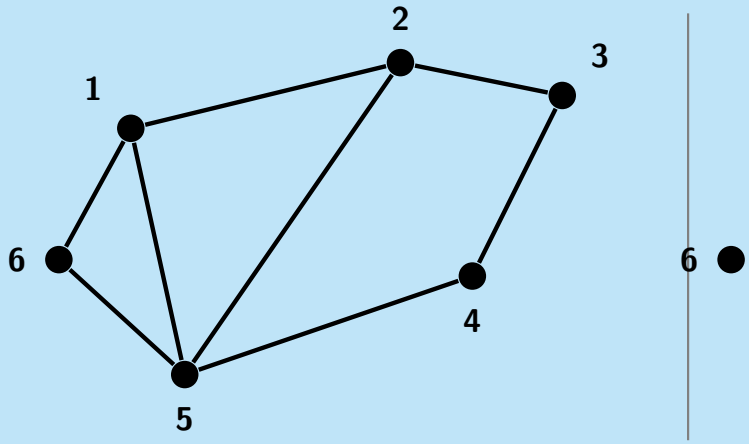


(c)

Corte de vértice e aresta

Definição

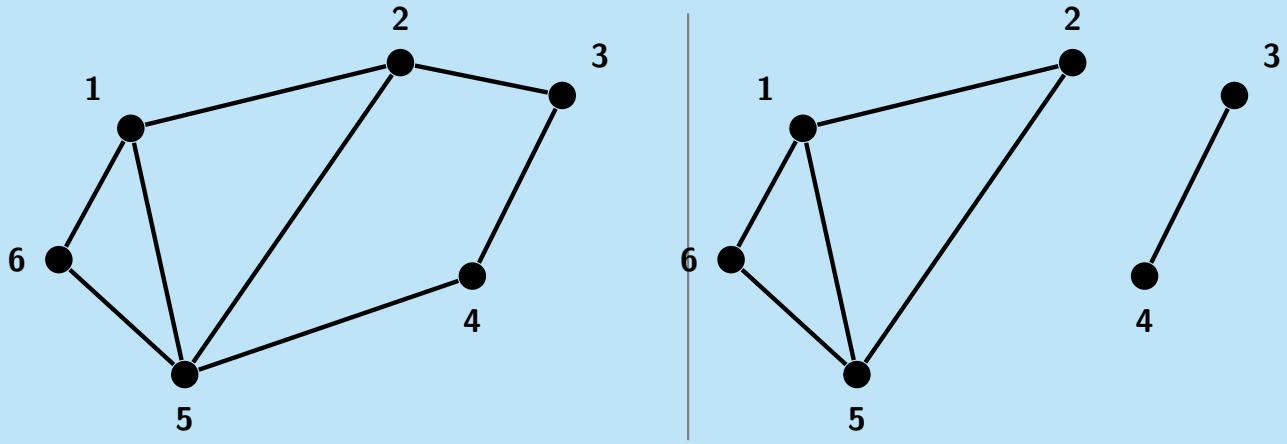
- **Corte de vértice** em $G(V, E)$ é um subconjunto **mínimo** de vértices V_1, com $V_1 \subseteq V$, tal que, se os vértices de V_1 forem removidos de G, o grafo fica desconexo ou trivial.



Corte de vértice e aresta

Definição

- **Corte de aresta** em $G(V, E)$ é um subconjunto **mínimo** de arestas E_1, com $E_1 \subseteq E$, tal que, se as arestas de E_1 forem removidas de G, o grafo fica desconexo.

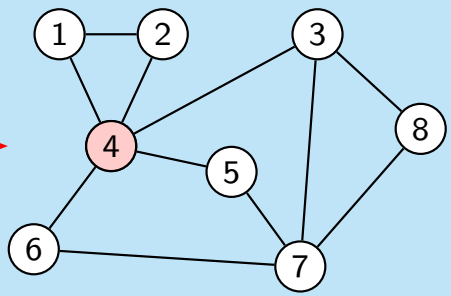


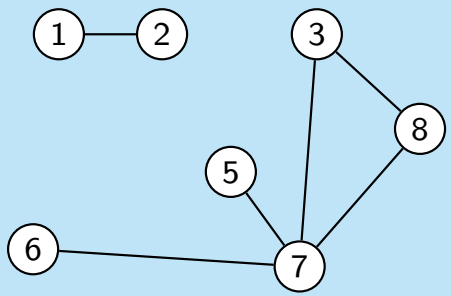
Vértice de corte

Definição

- Um vértice v_j de um grafo G é dito **vértice de corte ou ponto de articulação** se sua remoção (juntamente com as arestas a ele conectadas) provoca uma redução na conexidade, ou seja, torna o grafo desconexo.
  - Produz um grafo com mais componentes conexas que o grafo original.
  - $Componentes(G) < Componentes(G - v_j)$

Vértice de corte (articulação)



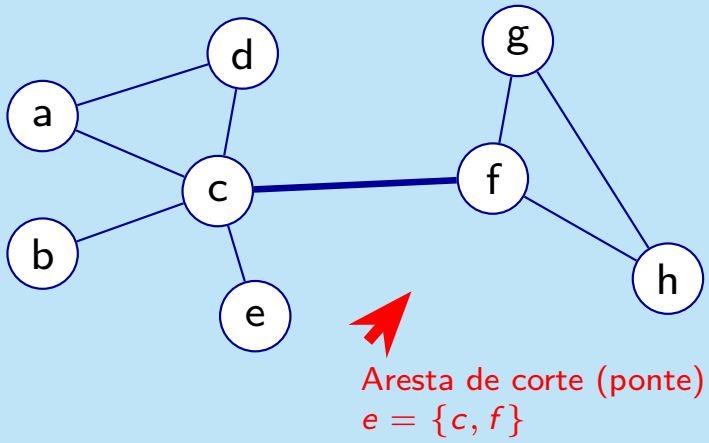


Remoção do vértice 4 criou duas componentes conexas

Aresta de Corte

Definição

- Uma aresta de G é dita **aresta de corte ou ponte** se, ao ser retirada de G, ele se torna desconexo.



Do Desenho ao Código

- Como traduzimos a ideia de um grafo para uma estrutura que o computador entenda? A escolha da representação é **crítica** porque afeta diretamente a eficiência (tempo e memória) dos nossos algoritmos.

As Três Grandes Representações

1. **Matriz de Adjacências:** ótima para verificar rapidamente se uma aresta existe.
2. **Lista de Adjacências:** a mais comum e eficiente em memória para grafos com poucas arestas.
3. **Matriz de Incidência:** mais especializada, útil para certas análises de rede.



Matriz de Adjacências

- Representamos o grafo como uma matriz quadrada M de dimensão $|V| \times |V|$.
- Se os vértices são numerados de 0 a $|V| - 1$, a entrada $M[i][j]$ da matriz informa sobre a aresta entre o vértice i e o vértice j.

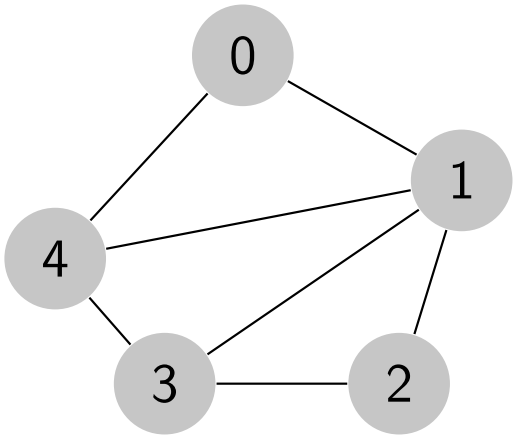
Regra (para grafos não-ponderados)

$$M[i][j] = \begin{cases} 1, & \text{se existe a aresta } (i, j) \\ 0, & \text{caso contrário} \end{cases}$$

- Para grafos ponderados, armazenamos o peso da aresta em vez de 1.
- Para grafos **não-direcionados**, a matriz é sempre **simétrica** ($M[i][j] = M[j][i]$).



Exemplo: Matriz de Adjacências



	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

Análise

- **Espaço:** $O(|V|^2)$. Ineficiente para grafos com poucas arestas (esparsos).
- **Verificar Aresta** (u, v)**:** $O(1)$. Excelente!
- **Listar Vizinhos de** u**:** $O(|V|)$. Precisa varrer uma linha inteira.

- A matriz de adjacências é **simétrica** para grafos não direcionados
- Dígrafo
  - Para (v_i, v_j), partindo de v_i e chegando em v_j, somente a posição M_{ij} será preenchida com 1
- Espaço de armazenamento das informações
  - $O(|V|^2)$
  - Indicado para **grafos densos**
    - Número de arestas é próximo de $|V|^2$
  - Desvantagem em **grafos esparsos**
    - Número de arestas é bem menor que $|V|^2$



Implementação em C - Matriz de adjacencias

Python Code

C Code

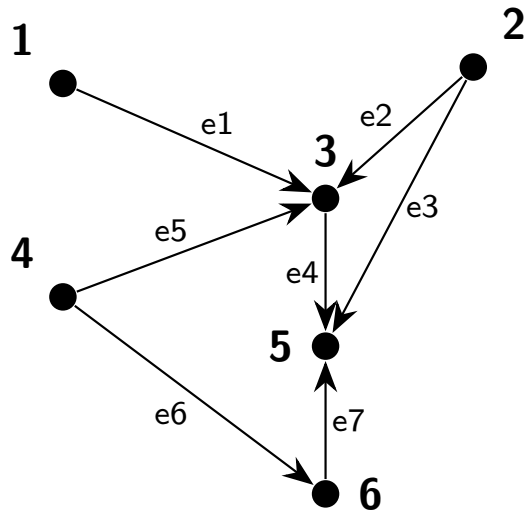


Definição

- Dado $G(V, E)$ com $|V|$ vértices e $|E|$ arestas, a matriz de incidência $M_{|V|\times|E|} = [b_{ij}]$ é definida por:

$$b_{ij} = \begin{cases} 1, & \text{se a aresta } j \text{ sai do vértice } i \\ -1, & \text{se a aresta } j \text{ entra no vértice } i \\ 0, & \text{caso contrário} \end{cases} \quad (1)$$

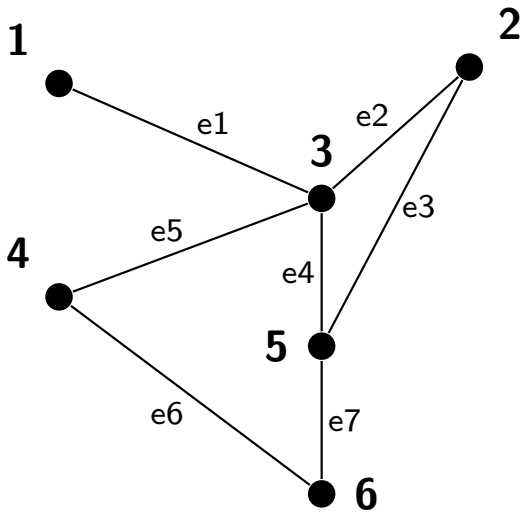




	e1	e2	e3	e4	e5	e6	e7
1	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0
3	−1	−1	0	1	−1	0	0
4	0	0	0	0	1	1	0
5	0	0	−1	−1	0	0	−1
6	0	0	0	0	0	−1	1

Dígrafo e sua Matriz de incidência





	e1	e2	e3	e4	e5	e6	e7
1	1	0	0	0	0	0	0
2	0	1	1	0	0	0	0
3	1	1	0	1	1	0	0
4	0	0	0	0	1	1	0
5	0	0	1	1	0	0	1
6	0	0	0	0	0	1	1

Grafo e sua Matriz de incidência

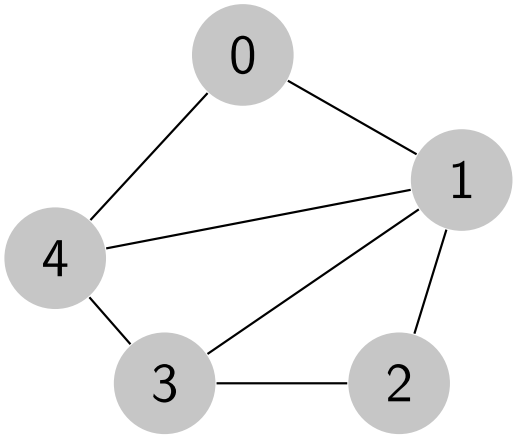


Lista de Adjacências

- A representação mais popular para a maioria das aplicações.
- Consiste em um array (ou vetor) de $|V|$ listas.
- Para cada vértice u, a entrada $Adj[u]$ armazena uma lista de todos os vértices v que são adjacentes a u.



Exemplo: Lista de Adjacências



$0 : \rightarrow [1, 4]$
$1 : \rightarrow [0, 2, 3, 4]$
$2 : \rightarrow [1, 3]$
$3 : \rightarrow [1, 2, 4]$
$4 : \rightarrow [0, 1, 3]$

Análise

- **Espaço:** $O(|V| + |E|)$. Muito eficiente para grafos esparsos.
- **Verificar Aresta** (u, v)**:** $O(deg(u))$. Precisa percorrer a lista de u.
- **Listar Vizinhos de** u**:** $O(deg(u))$. Excelente, o custo é proporcional ao número de vizinhos.

Implementação em C - Lista de adjacencias

Python Code

C Code



Introdução

- O objetivo é mostrar um **procedimento sistemático** para percorrer os vértices e as arestas de um grafo.
- **Marcar um vértice**
  - Evitar repetição
  - Distinguir dos vértices que ainda não foram processados
- Ideia geral
  - Dado $G(V, E)$ conexo e com todos os vértices desmarcados
  - Inicialmente, marque um vértice u qualquer. Nenhuma aresta $(u, v) \in E$ foi selecionada ainda
  - Ao selecionar a aresta (u, v), o vértice v é marcado



- Algoritmo de busca em profundidade (*Depth-First Search* - DFS)
- Algoritmo de busca em largura (*Breadth-First Search* - BFS)



- Critério da busca em largura para escolher o próximo vértice marcado a ser explorado:
  - *dentre os vértices marcados e incidentes a alguma aresta ainda não explorada, escolher aquele que foi alcançado primeiro na busca*
- Arquétipo de muitos algoritmos importantes
  - Algoritmo de Dijkstra.
  - Algoritmo de árvore geradora mínima de Prim.



- **Ideia Central:** explorar o grafo "camada por camada" a partir de um vértice de origem.
- Primeiro, visita todos os vizinhos diretos (distância 1), depois os vizinhos dos vizinhos (distância 2), e assim por diante.

Como Funciona?

Utiliza uma **Fila (Queue)** para controlar a ordem de visitação.

1. Comece em um vértice s, marque-o como visitado e coloque-o na fila.
2. Enquanto a fila não estiver vazia:
   - 2.1 Retire um vértice u da frente da fila.
   - 2.2 Para cada vizinho v de u que ainda não foi visitado:
     - Marque v como visitado.
     - Coloque v no final da fila.

Principal Aplicação

A BFS garante encontrar o **caminho mais curto** (em número de arestas) entre a origem s e qualquer outro vértice em um grafo **não-ponderado**.



- Dado $G(V, E)$ conexo e um vértice de origem **s**, a busca em largura explora as arestas de G até alcançar todos os vértices acessíveis a partir de **s**.
- Calcula a menor distância, em número de arestas, de **s** até cada vértice de $G(V, E)$
- O nome vem do fato de que ela descobre todos os vértices que estão a uma distância **k** de **s** antes de descobrir os que estão a uma distância $\mathbf{k+1}$
  - A fronteira entre vértices descobertos e não descobertos avança de maneira uniforme ao longo de toda a sua largura



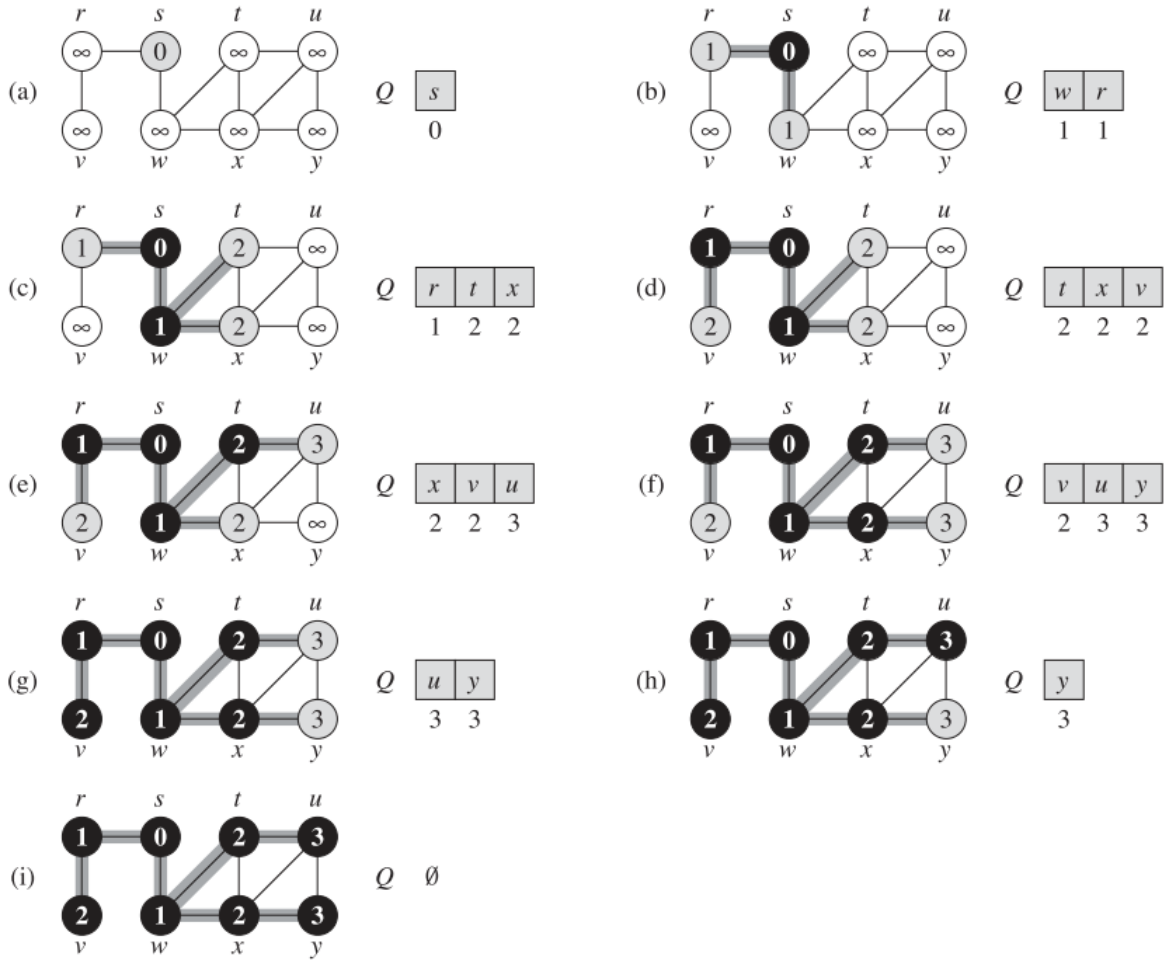
- Produz uma **árvore** com raiz **s**
- Computa o *caminho mais curto* de **s** até **v**
  - Número mínimo de arestas
- Aplicada em grafos direcionados ou não direcionados



Algoritmo

- Cada vértice é colorido de **branco, cinza ou preto**.
- Todos os vértices são inicializados como **branco**
- Quando um vértice é descoberto pela primeira vez, ele se torna **cinza**
- Vértices **cinza** e **preto** já foram descobertos, mas são distinguidos para assegurar que a busca ocorra em largura.
- Se $(u, v) \in E$ e o vértice u é preto, então o vértice v tem que ser cinza ou preto
- Vértices **cinza** podem ter vértices adjacentes **brancos**, e eles representam a **fronteira** entre vértices descobertos e não descobertos

```
BFS(G, s)
 1 for cada vértice u ∈ V[G] – {s
 2       do cor[u] ← BRANCO
 3          d[u] ← ∞
 4          α[u] ← NIL
 5 cor[s] ← CINZA
 6 d[s] ← 0
 7 α[s] ← NIL
 8 Q ← 0
 9 ENQUEUE(Q, s)
10   while Q ≠ 0
11     do u ← DEQUEUE(Q)
12       for cada v ← Adj[u]
13          do if cor[v] = BRANC
14                then cor[v] ← CI
15                     d[v] ← d[u
16                     π[v] ← u
17                     ENQUEUE
18       cor[u] ← PRETO
```



Execução do algoritmo busca em largura

Análise

- O teste da linha 13 assegura que cada vértice é colocado na fila no máximo uma vez.
- As operações de enfileirar e desenfileirar têm custo O(1), logo o custo total com a fila é $O(|V|)$.
- Cada lista de adjacências é percorrida no máximo uma vez, quando o vértice é desenfileirado.
- Como a soma dos comprimentos de todas as listas de adjacências é $\Theta(|E|)$, o tempo total gasto com essas listas é $O(|E|)$
- Tempo total da BFS: $O(|V| + |E|)$

```
BFS(G, s)
 1 for cada vértice u ∈ V[G] – {s
 2       do cor[u] ← BRANCO
 3          d[u] ← ∞
 4          α[u] ← NIL
 5 cor[s] ← CINZA
 6 d[s] ← 0
 7 α[s] ← NIL
 8 Q ← 0
 9 ENQUEUE(Q, s)
10  while Q ≠ 0
11    do u ← DEQUEUE(Q)
12       for cada v ← Adj[u]
13          do if cor[v] = BRANC
14                then cor[v] ← CI
15                     d[v] ← d[u
16                     π[v] ← u
17                     ENQUEUE
18       cor[u] ← PRETO
```

Implementação - Busca em Largura

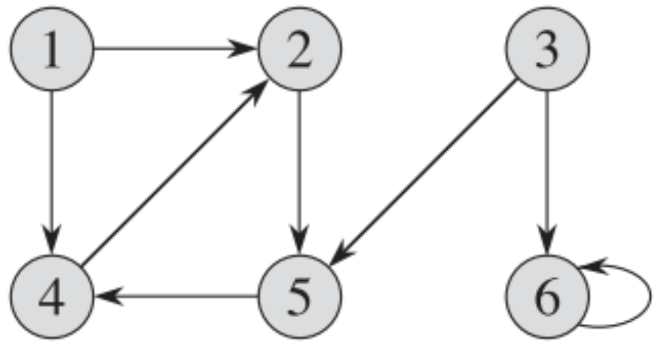
Python Code

C Code



Busca em Largura - Exercício I

- Mostre os valores de **d** e da fila que resultam da execução da busca em largura sobre o grafo ao lado, usando o vértice **3** como origem



- A estratégia é buscar o mais fundo possível no grafo sempre que der.
- As arestas são exploradas a partir do vértice **v** descoberto mais recentemente que ainda possui arestas não exploradas saindo dele.
- Quando todas as arestas adjacentes a **v** tiverem sido exploradas, a busca retrocede para explorar as arestas que saem do vértice a partir do qual **v** foi descoberto.
- O algoritmo é a base para muitos outros algoritmos importantes, como a verificação de grafos acíclicos, a ordenação topológica e a identificação de componentes fortemente conectados.



Ideia Central

- Para acompanhar o progresso do algoritmo, cada vértice é **colorido** de branco, cinza ou preto
- Todos os vértices são inicializados como branco
- Quando um vértice é descoberto pela primeira vez, ele se torna **cinza**, e passa a **preto** quando sua lista de adjacentes tiver sido completamente examinada.
- **Carimbo de Tempo**
  - $d[v]$: tempo de descoberta
  - $f[v]$: tempo de término do exame da lista de adjacentes de **v**
  - Esses registros são inteiros entre 1 e $2|V|$, pois existe um evento de descoberta e um evento de término para cada um dos $|V|$ vértices



Ideia Central

- Sempre que um vértice **v** é descoberto durante a varredura da lista de adjacências de um vértice já descoberto **u**
  - o algoritmo registra esse evento definindo o **predecessor** de **v**, $\pi[\mathbf{v}]$
  - $\pi[\mathbf{v}] = u$

    subgrafo predecessor $G_\pi = (V, E_\pi)$
    - $E_\pi = \{(\pi[v], v) : v \in V \text{ e } \pi[v] \neq \text{NIL}\}$
    - A DFS forma várias árvores (uma floresta), porque a busca pode ser repetida a partir de diversas origens.



Algoritmo - Busca em Profundidade

```
DFS(G)
1  for each vertex u ∈ G.V
2      u.color = WHITE
3      u.π = NIL
4  time = 0
5  for each vertex u ∈ G.V
6      if u.color == WHITE
7          DFS-VISIT(G, u)

DFS-VISIT(G, u)
1  time = time + 1          // white vertex u has just been discovered
2  u.d = time
3  u.color = GRAY
4  for each v ∈ G.Adj[u]    // explore edge (u, v)
5      if v.color == WHITE
6          v.π = u
7          DFS-VISIT(G, v)
8  u.color = BLACK          // blacken u; it is finished
9  time = time + 1
10 u.f = time
```

```
DFS(G)
1 for cada vértice u ← V[G]
2     do cor[u] ← BRANCO
3        π[u] ← NIL
4 tempo ← 0
5 for cada vértice u ∈ V[G]
6     do if cor[u] = BRANCO
7        then DFS-VISIT(u)

DFS-VISIT(u)
1 cor[u] ← CINZA          ▷ Branco, o vértice u acabou de ser
2 tempo ← tempo + 1
3 d[u] ← tempo
4 for cada v ∈ Adj[u]     ▷ Explora a aresta (u, v).
5     do if cor[u] = BRANCO
6        then π[v] ← u
7           DFS-VISIT(v)
8 cor[u] ← PRETO          ▷ Enegrece u; terminado.
9 f[u] ← tempo ← tempo + 1
```



Implementação em C - Busca em Profundidade

```c
#include <stdio.h>
#include <stdbool.h>
#define MAX_VERTICES 100

typedef struct {
    int matriz[MAX_VERTICES][MAX_VERTICES];
    int numVertices;
} Grafo;
```



Implementação em C - Busca em Profundidade

```c
void inicializarGrafo(Grafo* grafo, int numVertices) {
    int i, j;

    grafo->numVertices = numVertices;

    for (i = 0; i < numVertices; i++) {
        for (j = 0; j < numVertices; j++) {
            grafo->matriz[i][j] = 0;
        }
    }
}
```



Implementação em C - Busca em Profundidade

```c
void adicionarAresta(Grafo* grafo, int origem, int destino) {
    grafo->matriz[origem][destino] = 1;
    // Se for um grafo nao direcionado, descomente a linha abaixo
    // grafo->matriz[destino][origem] = 1;
}
```



Implementação em C - Busca em Profundidade

```
void DFSVisitar(Grafo* grafo, int vertice, bool visitado[]) {
    visitado[vertice] = true;
    printf("%d ", vertice);

    int i;
    for (i = 0; i < grafo->numVertices; i++) {
        if (grafo->matriz[vertice][i] == 1 && !visitado[i]) {
            DFSVisitar(grafo, i, visitado);
        }
    }
}
```



Implementação em C - Busca em Profundidade

```c
void buscaEmProfundidade(Grafo* grafo, int verticeInicial) {
    bool visitado[MAX_VERTICES];
    int i;

    for (i = 0; i < grafo->numVertices; i++) {
        visitado[i] = false;
    }

    DFSVisitar(grafo, verticeInicial, visitado);
}
```



Implementação em C - Busca em Profundidade

```c
int main() {
    Grafo grafo;
    int numVertices = 5;
    inicializarGrafo(&grafo, numVertices);
    adicionarAresta(&grafo, 0, 1);
    adicionarAresta(&grafo, 0, 4);
    adicionarAresta(&grafo, 1, 2);
    adicionarAresta(&grafo, 1, 3);
    adicionarAresta(&grafo, 1, 4);
    adicionarAresta(&grafo, 2, 3);
    adicionarAresta(&grafo, 3, 4);

    int verticeInicial = 0;
    printf("DFS a partir do vertice %d: ", verticeInicial);
    buscaEmProfundidade(&grafo, verticeInicial);

    return 0;
}
```



```
class DFS:
    def __init__(self,grafo):
        self.grafo= grafo;

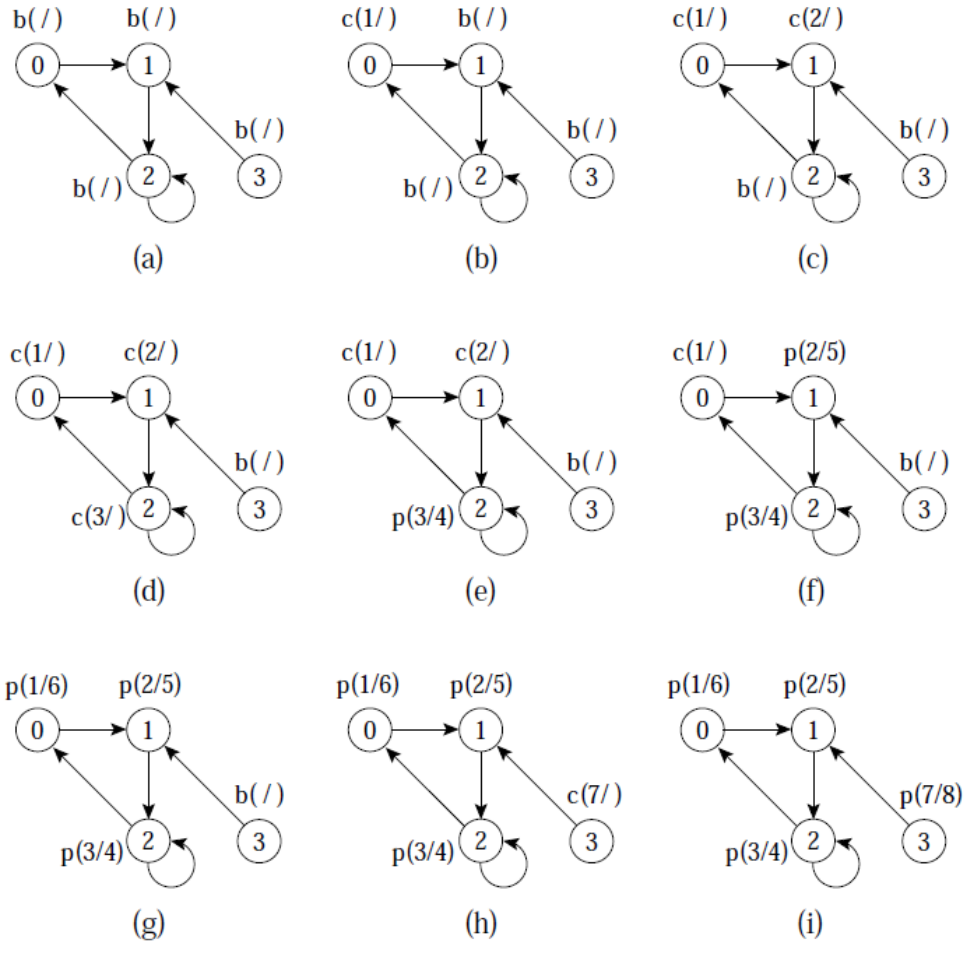
    def run(self):
        self.cor = ['BRANCO'] * (len(self.grafo))
        self.f = [0] * (len(self.grafo))
        self.d= [0] * (len(self.grafo))
        self.time = 0
        for u in self.grafo.listVertices():
            if(self.cor[self.grafo.indexOfVertice(u)]=='BRANCO'):
                self.DFS_visit(self.grafo,u);
        for v,d,f in zip(self.grafo.listVertices(),self.d,self.f):
            print("%s (%s/%s)" % (v,d,f))

    def DFS_visit(self,grafo,u):
        self.time = self.time + 1
        self.d[self.grafo.indexOfVertice(u)] = self.time
        self.cor[self.grafo.indexOfVertice(u)] = 'CINZA'
        for vertice_adj in self.grafo.listAdjOf(u):
            v = self.grafo.indexOfVertice(vertice_adj)
            if self.cor[v] == 'BRANCO':
                self.DFS_visit(grafo,vertice_adj)

        self.cor[self.grafo.indexOfVertice(u)] = 'PRETO'
        self.time = self.time + 1
        self.f[self.grafo.indexOfVertice(u)] = self.time
```



Algoritmo - Busca em Profundidade



Implementação em Python

```
if __name__ == '__main__':
    g = GrafoMatrix()
    for v in ["0","1", "2", "3"]:
        g.add_vertice(v)

    g.add_aresta("0", "1")
    g.add_aresta("1", "2")
    g.add_aresta("2", "0")
    g.add_aresta("2", "2")
    g.add_aresta("3", "1")
    print("Seguinte Percurso usando DFS")
    dfs = DFS(grafo=g)
    dfs.run()
```

Resultado

Seguinte Percurso
usando DFS
0(1/6)
1(2/5)
2(3/4)
3(7/8)



Análise - Busca em Profundidade

- Os dois *loops* do procedimento DFS gastam tempo $\Theta(V)$
- O procedimento *DFS_VISIT* é chamado exatamente uma vez para cada vértice $v \in V$, já que ele só é chamado para vértices brancos e sua primeira ação é pintá-los de cinza.
- No loop das linhas 4-7 de $DFS_VISIT(v)$, o corpo é executado $|Adj[v]|$ vezes
  - O custo total de tempo é $\sum_{v \in V} |Adj[v]| = \Theta(E)$
- Tempo total: $\Theta(V + E)$



Classificação das Arestas - Busca em Profundidade

A classificação das arestas pode ser útil para derivar outros algoritmos

Arestas de árvore são as arestas de uma árvore de busca em profundidade. A aresta $(\mathbf{u}, \mathbf{v})$ é uma aresta de árvore se $\mathbf{v}$ foi descoberto pela primeira vez ao percorrer a aresta $(\mathbf{u}, \mathbf{v})$

Arestas de retorno conectam um vértice $\mathbf{u}$ a um antecessor $\mathbf{v}$ em uma árvore de busca em profundidade (inclui *self-loops*).

Arestas de avanço são as arestas $(\mathbf{u}, \mathbf{v})$ que não pertencem à árvore de busca, mas conectam um vértice $\mathbf{u}$ a um descendente $\mathbf{v}$ nessa árvore.

Arestas de cruzamento são todas as demais. Podem conectar vértices na mesma árvore de busca em profundidade ou em duas árvores diferentes



Busca em Profundidade - Trabalho I

- A busca em profundidade pode ser usada para verificar se um grafo é acíclico ou se contém um ou mais ciclos.
- Se uma aresta de retorno é encontrada durante a busca em $G(V, E)$, então o grafo tem ciclo
- Um grafo direcionado $G(V, E)$ é acíclico se e somente se a busca em profundidade em $G(V, E)$ não apresentar arestas de retorno
- O algoritmo pode ser alterado para descobrir arestas de retorno. Basta verificar se um vértice **v** adjacente a **u** está cinza na primeira vez em que a aresta $(\mathbf{u}, \mathbf{v})$ é percorrida

1. Faça uma implementação para verificar se um $G(V, E)$ é acíclico.



O Problema do Caminho Mínimo

- **Objetivo:** dado um grafo **ponderado** com pesos não-negativos, um vértice de origem s e um de destino t, encontrar o caminho de s a t com a menor soma de pesos das arestas.
- Esta é uma das aplicações mais famosas e úteis de grafos, sendo a base para sistemas de GPS e roteamento de redes.

Algoritmo de Dijkstra

- É um algoritmo **guloso (greedy)** que resolve o problema do caminho mínimo de uma única origem para todos os outros vértices.
- **Ideia principal:** é uma generalização da BFS. Em vez de uma fila comum, usa uma **Fila de Prioridade (Priority Queue)** para escolher sempre, como próximo vértice a ser explorado, aquele com a menor distância *conhecida* da origem.
- **Restrição Crucial:** o algoritmo de Dijkstra **não funciona corretamente** se o grafo tiver arestas com pesos negativos.



Como Dijkstra Funciona (Intuitivamente)

1. Crie um conjunto de "distâncias" para todos os vértices, inicializando a distância da origem s como 0 e a de todos os outros como infinito.
2. Adicione a origem s a uma fila em que a prioridade é a distância.
3. Mantenha um conjunto de vértices já "finalizados" (cujo caminho mínimo já foi encontrado), inicialmente vazio.
4. Enquanto a fila de prioridade não estiver vazia:
   - 4.1 Extraia da fila o vértice u com a menor distância. Adicione u ao conjunto de finalizados.
   - 4.2 Para cada vizinho v de u:
     - Calcule uma nova distância para v passando por u: $dist(s, u) + peso(u, v)$.
     - Se essa nova distância for menor que a distância atual de v, atualize a distância de v e sua prioridade na fila. Este passo é chamado de **relaxamento (relax)**.

- Ao final, o array de distâncias conterá o custo do caminho mínimo de s até cada um dos outros vértices.



Problema dos Caminhos Mais Curtos

- Modelagem
  - $G = (V, E)$: grafo direcionado ponderado.
  - Função peso $\mathbf{w} : E \to \mathbb{R}$
    - $w(u, v)$: peso de cada aresta
  - O **peso** de um caminho $\mathbf{p} = \langle \mathbf{v_0}, \mathbf{v_1}, \ldots, \mathbf{v_k} \rangle$ é a soma dos pesos de suas arestas:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

- Definimos o **peso do caminho mais curto** de u até v por:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \overset{p}{\rightsquigarrow} v\} & \text{se existir caminho de } u \text{ a } v \\ \infty, & \text{caso contrário} \end{cases} \quad (2)$$

- O **caminho mais curto** do vértice **u** ao vértice **v** é, então, qualquer caminho **p** com peso $\mathbf{w(p)} = \delta(\mathbf{u}, \mathbf{v})$



Problema dos Caminhos Mais Curtos

- Modelagem de Problema
  - Saber a rota mais curta entre Belém e Parauapebas
  - V: Estações
  - E: Segmentos de Estrada
  - w: **distâncias** entre as estações
    - **Medidas**: quantidade que se acumula linearmente ao longo de um caminho e que se deseja minimizar
  - G(V,E): Mapa rodoviário
- O algoritmo de Busca em Largura será nossa referência.
- **Caminhos mais curtos a partir de uma origem**
  - Dado um grafo ponderado $G = (V, E)$, desejamos obter o caminho mais curto de um vértice de origem $s \in V$ até cada $v \in V$



Representação de Caminhos Mais Curtos

- Os caminhos mais curtos podem ser representados pela variável **predecessor** π
- Para cada vértice $v \in V$, o predecessor $\pi[\mathbf{v}]$ é outro vértice ou *Null*
- O algoritmo atribui aos predecessores os rótulos de uma cadeia de vértices que começa em v e anda para trás ao longo de um caminho mais curto de s até v
- Em um passo intermediário, os valores de $\pi[\mathbf{v}]$ não indicam necessariamente caminhos mais curtos
- Ao final do processamento, porém, os predecessores formam uma árvore de caminhos mais curtos definidos **em termos dos pesos das arestas** de G, e não do **número de arestas**.



Algoritmo de Dijkstra

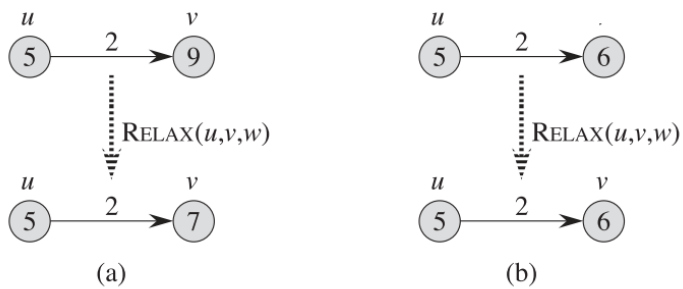
- Mantém um conjunto S de vértices cujos caminhos mais curtos a partir da origem já são conhecidos.
- Produz uma árvore de caminhos mais curtos de um vértice de origem s para todos os vértices alcançáveis a partir de s.
- Utiliza **Relaxamento**
  - Para cada $v \in V$, o atributo $\mathbf{d}[\mathbf{v}]$ é um limite superior para o peso de um caminho mais curto da origem s até v
  - O vetor $\mathbf{d}[\mathbf{v}]$ contém uma estimativa de caminho mais curto
- O primeiro passo do algoritmo é inicializar os predecessores e as estimativas de caminhos mais curtos
  - Método *INITIALIZE-SINGLE-SOURCE(G,s)*



Relaxamento

- *O relaxamento de uma aresta* (u, v) *consiste em verificar se é possível melhorar o melhor caminho até* **v** *obtido até o momento passando por* **u**.
  - Uma etapa de relaxamento pode diminuir $d[v]$ e atualizar o campo $\pi[v]$.
  - No Dijkstra, cada aresta é relaxada somente 1 vez.

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2        v.d = u.d + w(u, v)
3        v.π = u
```



Método *Relax*

Algoritmo de Dijkstra

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2       v.d = ∞
3       v.π = NIL
4  s.d = 0

RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2       v.d = u.d + w(u, v)
3       v.π = u

DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5       u = EXTRACT-MIN(Q)
6       S = S ∪ {u}
7       for each vertex v ∈ G.Adj[u]
8            RELAX(u, v, w)
```

```
INITIALIZE-SINGLE-SOURCE(G, s)
1 for cada vértice v ∈ V[G]
2    do d[v] ← ∞
3       π[v] ← NIL
4 d[s] ← 0

RELAX(u, v, w)
1 if d[v] > d[u] + w(u, v)
2   then d[v] ← d[u] + w(u, v)
3        π[v] ← u

DIJKSTRA(G, w, s)
1 INITIALIZE-SINGLE-SOURCE(G, s)
2 S ← ∅
3 Q ← V[G]
4 while Q ≠ ∅
5   do u ← EXTRACT-MIN(Q)
6      S ← S ∪ {u}
7      for cada vértice v ∈ Adj[u]
8           do RELAX(u, v, w)
```

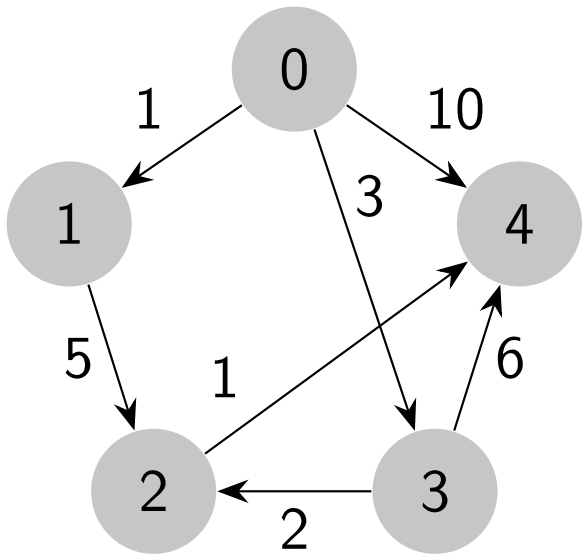


Dijkstra Passo a Passo: Início no Vértice 0

- O conjunto de vértices visitados **S** começa vazio.
- A Fila de Prioridade **Q** contém todos os vértices.
- A distância $d[0]$ da origem é **0**.
- Todas as outras distâncias $d[v]$ são inicializadas com ∞.

Vértice	**d[v]**	π**[v]**
0	0	NULL
1	∞	NULL
2	∞	NULL
3	∞	NULL
4	∞	NULL

$\mathbf{S} = \{\}$

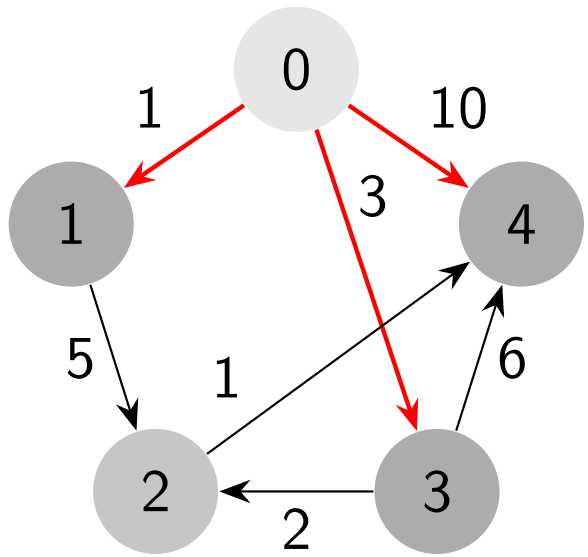


Dijkstra Passo a Passo: Início no Vértice 0

- Extraímos de Q o vértice com menor distância: **0**.
- Adicionamos **0** ao conjunto S.
- **Relaxamos** as arestas que saem de 0:
  - (0,1): $d[1] = \min(\infty, 0 + 1) = 1$
  - (0,3): $d[3] = \min(\infty, 0 + 3) = 3$
  - (0,4): $d[4] = \min(\infty, 0 + 10) = 10$

Vértice	d[v]	π[v]
0	0	NULL
1	1	0
2	∞	NULL
3	3	0
4	10	0

$\mathbf{S} = \{\mathbf{0}\}$

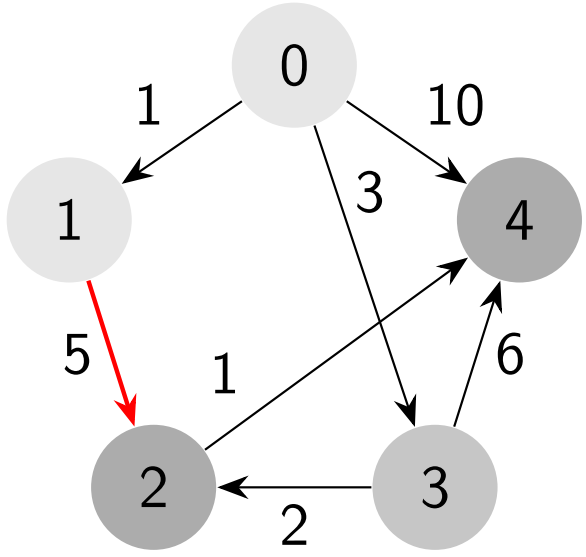


Dijkstra Passo a Passo: Início no Vértice 0

- Próximo vértice de Q com menor distância: **1** (d=1).
- Adicionamos **1** ao conjunto S.
- **Relaxamos** a aresta (1,2):
  - $d[2] = \min(\infty, d[1] + 5) = \min(\infty, 1 + 5) = 6$

Vértice	d[v]	π[v]
0	0	NULL
1	1	0
2	6	1
3	3	0
4	10	0

$$\mathbf{S} = \{\mathbf{0}, \mathbf{1}\}$$

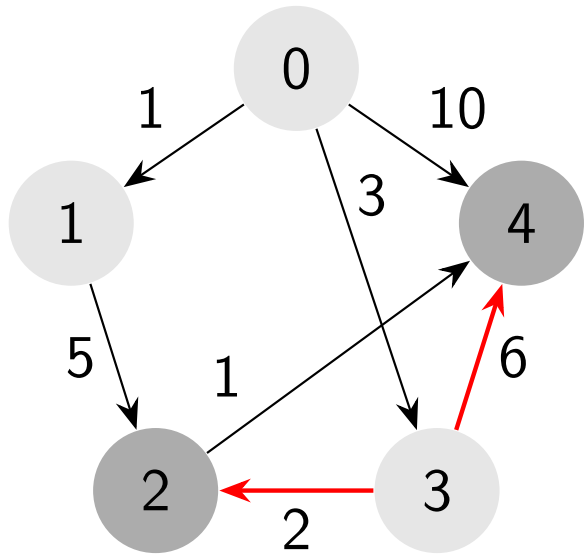


Dijkstra Passo a Passo: Início no Vértice 0

- Próximo vértice de Q: **3** (d=3).
- Adicionamos **3** ao conjunto S.
- **Relaxamos** as arestas que saem de 3:
  - (3,2): $d[2] = \min(6, d[3] + 2) = \min(6, 3 + 2) = 5$
  - (3,4): $d[4] = \min(10, d[3] + 6) = \min(10, 3 + 6) = 9$

Vértice	d[v]	π[v]
0	0	NULL
1	1	0
2	5	3
3	3	0
4	9	3

$\mathbf{S} = \{\mathbf{0}, \mathbf{1}, \mathbf{3}\}$

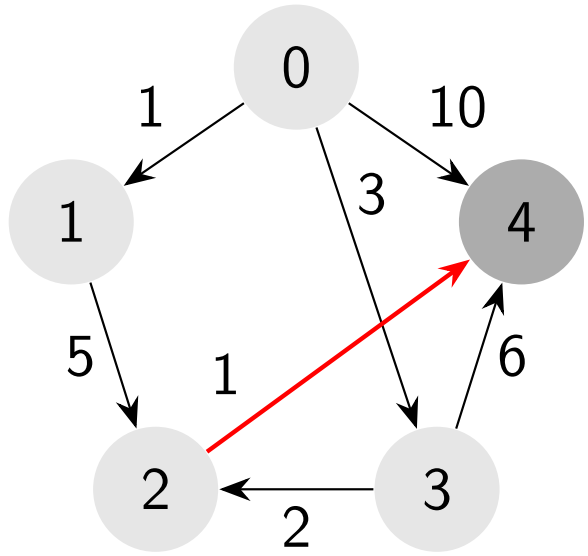


Dijkstra Passo a Passo: Início no Vértice 0

- Próximo vértice de Q: **2** (d=5).
- Adicionamos **2** ao conjunto S.
- **Relaxamos** a aresta (2,4):
  - $d[4] = \min(9, d[2] + 1) = \min(9, 5 + 1) = 6$

Vértice	**d[v]**	π**[v]**
0	0	NULL
1	1	0
2	5	3
3	3	0
4	6	2

$$\mathbf{S} = \{\mathbf{0}, \mathbf{1}, \mathbf{3}, \mathbf{2}\}$$

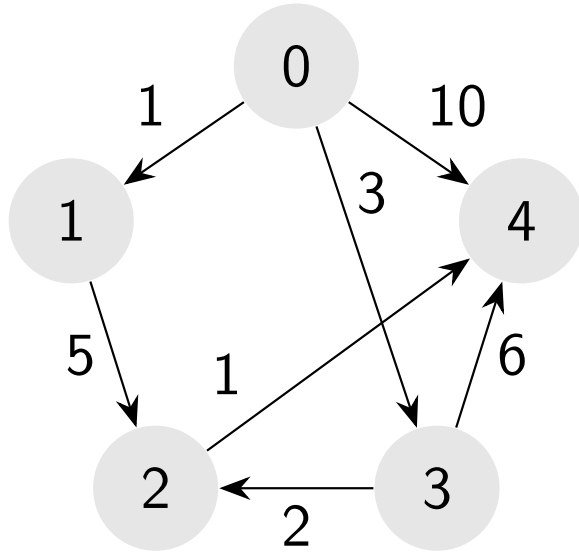


Dijkstra Passo a Passo: Início no Vértice 0

- Último vértice de Q: **4** (d=6).
- Adicionamos **4** ao conjunto S.
- Não há arestas para relaxar, pois não há vizinhos fora de S.
- A fila Q fica vazia. O algoritmo termina.

Vértice	d[v]	π[v]
0	0	NULL
1	1	0
2	5	3
3	3	0
4	6	2

$$\mathbf{S} = \{\mathbf{0}, \mathbf{1}, \mathbf{3}, \mathbf{2}, \mathbf{4}\}$$



Algoritmo de Dijkstra

- **Invariante:** no início de cada iteração do loop *while*, a fila de prioridade (Heap) **Q** contém exatamente os vértices de $V - S$.
- A cada iteração do *while*, um vértice **u** é extraído da fila e adicionado ao conjunto **S**, o que mantém o invariante.
- **EXTRACT-MIN** obtém o vértice **u** com o menor caminho estimado até o momento e o adiciona ao conjunto **S**.
- No loop das linhas 7-8, a operação de relaxamento é realizada sobre cada aresta $(\mathbf{u}, \mathbf{v})$ que sai do vértice **u**



Algoritmo de Dijkstra

- Para selecionar a nova aresta de forma eficiente, todos os vértices que ainda não estão na árvore de caminhos mais curtos ficam na fila **Q**, ordenada pelo campo **d**.
- Para cada vértice **v**, **d**[**v**] é o caminho mais curto obtido até o momento entre o vértice raiz e **v**.
- A fila guarda os vértices, mas a sua ordem é dada pelas estimativas armazenadas no arranjo **d**[**v**]. Por isso, dizemos que a fila é indireta.



Algoritmo de Dijkstra

- O algoritmo usa uma estratégia **gulosa**: escolhe sempre o vértice mais leve (o mais perto) em $V - S$ para adicionar ao conjunto solução **S**
- O algoritmo de Dijkstra sempre obtém os caminhos mais curtos, pois cada vez que um vértice u é adicionado ao conjunto **S** temos $d[u] = \delta(s, u)$.

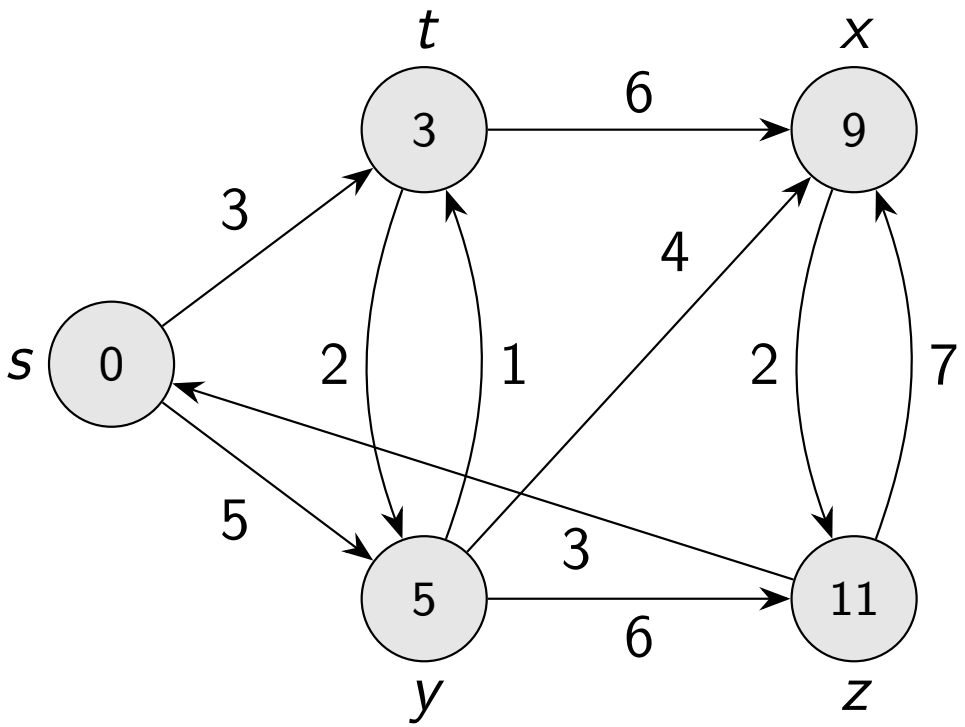


Algoritmo de Dijkstra - Trabalho II

1. Execute o algoritmo de Dijkstra sobre o grafo orientado do slide 91, primeiro usando o vértice **s** como origem e depois usando o vértice **z** como origem. Imprima os valores **d** e os vértices do conjunto **S** após cada iteração do loop *while*
2. Dado um dígrafo D sem ciclos direcionados, formule um algoritmo para encontrar o comprimento do maior caminho em D. A matriz de distâncias W de D é formada por distâncias não negativas.



Algoritmo de Dijkstra - Trabalho II



Referências I


Lee K.D., Hubbard S. (2015) Trees. In: Data Structures and Algorithms with Python. Undergraduate Topics in Computer Science. Springer, Cham. Retrieved from https://doi.org/10.1007/978-3-319-13072-9_6


Hubbard, J. (2007). Schaum's Outline of Data Structures with Java. Retrieved from http://www.amazon.com/Schaums-Outline-Data-Structures-Java/dp/0071476989


Cormen, T. H., Leiserson, C. E., & Stein, R. L. R. E. C. (2012). Algoritmos: teoria e prática. Retrieved from https://books.google.com.br/books?id=6iA4LgEACAAJ.


Ascencio, Ana Fernanda Gomes. Estrutura de dados: Algoritmos, análise da complexidade e implementações em Java e C++. São Paulo: Pearson Prentice Hall, 2010.



Referências II


Donald E. Knuth. Sorting and Searching, volume 3 of The Art of Computer Programming. Addison-Wesley, 1973. Second edition, 1998


Szwarcfiter, Jayme Luiz. Estruturas de dados e seus algoritmos / Jayme Luiz Szwarcfiter, Lilian Markenzon. 3.ed. [Reimpr.]. - Rio de Janeiro : LTC, 2015.


Learning about defaultdict in Python. Retrieved from `https://www.educative.io/edpresso/learning-about-defaultdict-in-python`.


How to implement a graph in Python. Retrieved from `https://www.educative.io/edpresso/how-to-implement-a-graph-in-python`.





Grafos

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br





[1]Faculdade de Sistemas de Informação - UFPA/CUTINS

27 de outubro de 2026